

**Examen écrit terminal
Semestre 6
Deuxième session
2021/2022**

Licence 3

EC : INFO0601

Intitulé : Systèmes d'exploitation : concepts avancés

Responsable : Cyril RABAT

Durée de l'épreuve : 2h00

Nombre de page(s) : 6

RAPPEL

Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur ne doit pas être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.

Sujet : M. Cyril Rabat (20 points)

Exercice 1 (Pixel War - 10 pts)

Nous souhaitons développer une application réseau qui permet de réaliser une fresque collective en pixels. Cette dernière fait 10000 par 10000 pixels et est stockée dans un fichier binaire, ligne par ligne, chaque pixel correspondant à 3 `unsigned char` (nous supposons l'existence de la structure `pixel_t`).

L'application repose sur un serveur UDP et des clients. Les clients peuvent effectuer deux types de requête : la récupération d'une portion de la fresque de 20 par 20 pixels et la modification d'un pixel.

- 1°) Expliquez quelles données sont échangées dans l'application et donnez les structures C correspondantes.
- 2°) Écrivez la fonction `ouvrir` qui permet d'ouvrir le fichier contenant la fresque. Le nom du fichier est passé en paramètre. Si le fichier n'existe pas, il est créé (une fresque avec des pixels noirs (0,0,0) est alors créée). La fonction retourne le descripteur de fichier. La vérification de l'existence du fichier doit être effectuée à l'aide de la fonction `open` et de la gestion d'erreur.
- 3°) Nous supposons l'existence de la fonction `setpixel` qui prend en paramètres le descripteur de fichier, les coordonnées (x, y) du pixel et sa couleur (de type `pixel_t`). Elle modifie le pixel correspondant. Sans donner le code, expliquez ce que fait cette fonction en précisant les appels systèmes nécessaires.
- 4°) Donnez le code du serveur permettant d'initialiser la socket. Le numéro de port est fixé par une constante `PORT`.
- 5°) À votre avis, est-il nécessaire de mettre en place une solution distribuée pour répondre aux requêtes des clients ? Si oui, expliquez.
- 6°) Donnez la portion de code permettant au serveur de recevoir les requêtes et d'y répondre.
- 7°) Nous supposons qu'un client donné ne peut modifier un pixel que toutes les 5 secondes minimum. Proposez une solution permettant de résoudre cette nouvelle contrainte.

Tournez la page pour la suite du sujet

Exercice 2 (File de messages partagée (les IPC) - 10 pts)

Nous souhaitons mettre en place une architecture distribuée de communication entre des clients et un contrôleur à l'aide d'une file de messages partagée. Les clients peuvent déposer des messages qui sont lus par les autres clients ou le contrôleur. Seul le contrôleur peut supprimer les messages de la file.

Chaque client est caractérisé par son statut : un PID, la date de connexion et la date de dernière émission dans la file. Le nombre maximum de clients est fixé à l'aide d'une constante $MAX_CLT = 10$. La file de messages, représentée par un tableau, contient un nombre maximal d'entrées fixé par une constante $MAX_ENT = 20$. Chaque entrée est caractérisée par le PID de l'émetteur et un message (une chaîne de caractères de taille fixe spécifiée par la constante $MAX_MSG = 256$). La file de messages et les statuts des clients sont stockés dans un segment de mémoire partagée. La gestion de la concurrence est réalisée à l'aide de sémaphores.

Pour accéder à la file, chaque client doit être connecté : il doit d'abord trouver une place disponible parmi les statuts et spécifier son statut.

Pour rappel, une file circulaire de taille T représentée à l'aide d'un tableau utilise deux entiers : les indices de début (D) et de fin (F). Lorsque $F=D$, la file est vide. Lorsque $(D+1) \% T = F$, la file est pleine (en réalité, la file ne peut contenir que $T-1$ éléments). Le prochain élément à lire est situé dans la case dont l'indice est F (ensuite $F = (F+1) \% T$). Un nouvel élément doit être placé dans la case d'indice D (ensuite $D = (D+1) \% T$).

1°) Expliquez pourquoi une file de messages IPC ne serait pas appropriée dans le cadre de cette application.

2°) Donnez le contenu du segment de mémoire partagée, en précisant le type des données. Spécifiez les structures nécessaires (statut et message).

Le tableau de sémaphores et le segment de mémoire partagée sont créés et initialisés par le contrôleur. Nous supposons que la limitation du nombre de clients est gérée à l'aide d'un sémaphore. Lorsqu'un client démarre, si le nombre maximal de clients est atteint, il doit s'arrêter et ne pas resté bloqué. Dès qu'un client se connecte, il spécifie son statut dans le segment de mémoire partagée dans un emplacement libre. Il peut ensuite ajouter des messages dans la file ou les consulter.

3°) Donnez les sémaphores nécessaires pour la phase de connexion, ainsi que leur initialisation.

4°) Donnez toutes les étapes nécessaires du démarrage d'un client jusqu'à la spécification de son statut dans le segment de mémoire partagée. Vous préciserez tous les appels systèmes ainsi que les sémaphores nécessaires (en indiquant leur valeur d'initialisation).

Pour éviter que des messages ne soient perdus, il faut empêcher les clients d'envoyer un message si la file est pleine. Par contre, les autres clients et le contrôleur doit pouvoir continuer à lire les messages.

5°) Proposez une solution à l'aide de sémaphores, tout en limitant le nombre d'appels systèmes et en évitant l'attente active. Décrivez précisément le fonctionnement de votre solution (ajout et lecture des messages).

Annexes : quelques en-têtes de fonctions C

```
int accept(int sockfd, struct sockaddr *adresseClient, socklen_t *taille);
unsigned int alarm(unsigned int nb_secondes);
int atexit(void (*procedure)(void));
int bind(int fd, struct sockaddr *adresse, socklen_t longueur);
int close(int fd);
int connect(int sockfd, struct sockaddr *adresseServeur, socklen_t taille);
int execve(const char *fichier, char *const argv[], char *const envp[]);
void exit(int statut);
int fcntl(int fd, int commande, ...);
pid_t fork();
key_t ftok(char *chemin, int proj_id);
pid_t getpid();
pid_t getppid();
int getpeername(int sockfd, struct sockaddr *adresse, socklen_t *taille);
int getsockname(int sockfd, struct sockaddr *adresse, socklen_t *taille);
uint32_t htonl(uint32_t hote_long);
uint16_t htons(uint16_t hote_court);
const char *inet_ntop(int famille, const void *source, char *destination, socklen_t taille);
int inet_pton(int famille, const char *source, void *destination);
int kill(pid_t pid, int numero_signal);
int listen(int sockfd, int taille);
off_t lseek(int fd, off_t offset, int point_depart);
int lstat(const char *chemin, struct stat *tampon);
void *memcpy(void *destination, const void *source, size_t taille);
void *memset(void *tampon, int caractere, size_t nombre_elements);
int mkfifo(const char *nomFichier, mode_t mode);
int msgctl(key_t cle, int commande, struct msqid_ds *buf);
int msgget(key_t cle, int options);
int msgrcv(int msqid, struct msgbuf *msg, size_t taille, long type, int options);
int msgsnd(int msqid, struct msgbuf *msg, size_t taille, int options);
uint32_t ntohl(uint32_t reseau_long);
uint16_t ntohs(uint16_t reseau_court);
int open(const char *chemin, int options, mode_t mode);
int pause(void);
int pipe(int tube[2]);
ssize_t recvfrom(int sockfd, void *message, size_t taille, int flags,
                 struct sockaddr *adresse_source, socklen_t *taille_adresse);
ssize_t read(int fd, void *tampon, size_t taille);
int semctl(int semid, int semnum, int commande, union senum args);
int semget(key_t cle, int nbSems, int options);
int semop(int semid, struct sembuf *sops, unsigned nsops);
ssize_t sendto(int sockfd, const void *message, size_t taille, int flags,
               const struct sockaddr *adresse_dest, socklen_t taille_adresse);
void *shmat(int shmid, const void *shmaddr, int options);
int shmctl(key_t cle, int commande, struct shmid_ds *buf);
int shmdt(const void *shmaddr);
int shmget(key_t cle, int taille, int options);
int sigaction(int num, const struct sigaction *action, struct sigaction *old_action);
int sigaddset(sigset_t *ensemble, int numero_signal);
int sigdelset(sigset_t *ensemble, int numero_signal);
int sigemptyset(sigset_t *ensemble);
int sigfillset(sigset_t *ensemble);
int sigismember(sigset_t *ensemble, int numero_signal);
int sigpending(sigset_t *ensemble);
int sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int sigsuspend(const sigset_t *ensemble);
int sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int sigtimedwait(const sigset_t *set, siginfo_t *info, const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
int socket(int domaine, int type, int protocole);
int socketpair(int domaine, int type, int protocol, int sv[2]);
int unlink(const char *nomFichier);
pid_t wait(int *statut);
pid_t waitpid(pid_t pid, int *statut, int options);
ssize_t write(int fd, const void *tampon, size_t taille);
```

Constantes/macros associées à des paramètres ou des champs de structures

cle (msgget, shmget, semget) : IPC_PRIVATE
commande (fcntl) : F_GETFL, F_SETFL
commande (msgctl, shmctl) : IPC_RMID, IPC_STAT, IPC_SET
commande (semctl) : IPC_RMID, IPC_STAT, IPC_SET, IPC_GETVAL, IPC_GETALL, IPC_SETVAL, IPC_SETALL
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
domaine (socket, socketpair) : AF_LOCAL, AF_UNIX, AF_INET, AF_INET6, AF_PACKET
famille (inet_pton, inet_ntop) : AF_INET, AF_INET6
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
mode (open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (fcntl, open) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
options (msgget, semget, shmget) : IPC_CREAT, IPC_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (msgrcv) : IPC_NOWAIT, MSG_EXCEPT
options (msgsnd) : IPC_NOWAIT, MSG_NOERROR
options (shmat) : SHM_RDONLY, SHM_RND
options (waitpid) : WNOHANG, WUNTRACED
point_depart (lseek) : SEEK_SET, SEEK_CUR, SEEK_END
protocole (socket, socketpair) : IPPROTO_TCP, IPPROTO_UDP
sa_flags (champ de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
sa_handler (champ de struct sigaction) : SIG_IGN, SIG_DFL
sem_flag (champ de struct sembuf) : IPC_NOWAIT, SEM_UNDO
taille (inet_ntop) : INET_ADDRSTRLEN, INET6_ADDRSTRLEN
type (socket, socketpair) : SOCK_STREAM, SOCK_DGRAM

Macros sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG

Quelques structures C

<pre> struct sigaction { void (*sa_handler) (int); void (*sa_sigaction) (int, sigset_t sa_mask; int sa_flags; }; union senum { int val; struct semid_ds *buf; unsigned short *array; }; struct siginfo_t { int si_signo; int si_code; sigval_t si_value; pid_t si_pid; ... }; struct msqid_ds { struct ipc_perm msg_perm; time_t msg_stime; time_t msg_rtime; time_t msg_ctime; msgnum_t msgnum; msglen_t msg_qbytes; pid_t msg_lspid; pid_t msg_lrpid; }; </pre>	<pre> struct in_addr { uint32_t s_addr; }; struct timespec { long tv_sec; long tv_nsec; }; struct semid_ds { struct ipc_perm sem_perm; time_t sem_otime; time_t sem_ctime; unsigned short sem_nsems; }; struct shmid_ds { struct ipc_perm msg_perm; size_t shm_segsz; time_t shm_atime; time_t shm_dtime; time_t shm_ctime; pid_t shm_cpid; pid_t shm_lpid; shmatt_t shm_nattch; }; struct sockaddr_in { sa_family_t sin_family; in_port_t sin_port; struct in_addr sin_addr; }; </pre>	<pre> struct sembuf { unsigned short sem_num; short sem_op; short sem_flg; }; struct sockaddr_in6 { sa_family_t sin6_family; in_port_t sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; uint32_t sin6_scope_id; }; struct sockaddr_un { sa_family_t sun_family; char sun_path[108]; }; struct in_addr6 { unsigned char s_addr6[16]; }; union sig_val_t { int sival_int; void *sival_ptr; }; struct timeval { long tv_sec; long tv_usec; }; </pre>
---	---	--